Real Analysis Tutor — Developer's Manual

Methodology & tutor system: Don Kearney

2026-07-18

Table of contents

1 Overview

1.1 What this is

`real-analysis-tutor` is a Claude Code plugin (`.claude-plugin/plugin.json`) that tutors a learner through Real Analysis — Baby Rudin, Strichartz, and Alcock — using the V2.1 mastery methodology: Socratic questioning, misconception probing, a Theorem Ledger, spaced review, and optional Lean 4 verification. It is Socratic by architecture: the front-door skill (`skills/tutor/SKILL.md`) holds as its prime directive that the proof must come from the learner, and it never supplies a proof, a partial proof, or a single step of one (`skills/tutor/SKILL.md`, "Identity and prime directive").

Author: Don Kearney. The tutor system and the V2.1 methodology it operationalizes — the *Real Analysis Complete Study Guide* (Kearney, 2026) and the *Low-Friction Setup & Workflow Guide* (Kearney, 2026), vendored under `docs/source-guides/` — are Don Kearney's work and property (`README.md`, `CHANGELOG.md` 2.1.0). The plugin implements that methodology as 10 skills, 5 agents, and 3 hooks (`skills/`, `agents/`, `hooks/hooks.json`).

1.2 Who this manual is for

This manual is for developers and curriculum authors, not learners. The learner-facing guarantee stated in `README.md` under "That's it" is that a learner never needs to read anything in this repository beyond the two install commands: the tutor introduces itself, interviews the learner to pick a starting point, creates its own workspace, and offers Lean 4 setup only when needed. `README.md` stays install-only by design; this manual holds the "how" and "why" for anyone maintaining the plugin, auditing its pedagogy, or porting it to a new subject.

1.3 Quickstart

Two commands, run inside Claude Code (`README.md`):

```
/plugin marketplace add apollostream/real-analysis-tutor
/plugin install real-analysis-tutor@real-analysis-tutor
```

Then say hello. There is no third command and no configuration file to edit first.

1.4 First run vs. returning sessions

On a first run — no `.ra-tutor-workspace` marker findable upward from the current directory (`skills/tutor/SKILL.md`, "First-run branch") — the tutor runs the onboarding interview in `skills/tutor/reference/onboarding.md`: it makes the one promise (never hand over a proof), asks about proof experience to pick a `quick_start_profiles` entry from `curriculum/real-analysis/curriculum.yaml`, asks where to keep the study workspace (default `~/real-analysis-study`), and materializes it from `templates/workspace/` before Stage 1. On later sessions it instead runs the open ritual in `skills/tutor/reference/open-ritual.md`: reads saved state and due reviews through the engine (`engine/state.py`, `engine/scheduler.py`), greets by progress rather than praise, surfaces concepts due for review, and proposes an agenda before handing off to the six-stage loop (`skills/tutor/reference/six-stage-loop.md`).

1.5 The learner's journey

From install to the accumulating record of a semester's work:

1.6 Map of this manual

- Pedagogy — the frameworks behind the tutoring loop: Solow, Pólya, Bloom, and Alcock, and how responsibility for each is divided across the plugin.
- Architecture — the three-layer design (deterministic engine, judgment-layer agents, fresh-context sessions), the hooks, and anti-leakage safeguards.
- Generality — how much of the plugin is a general tutoring engine versus Real Analysis-specific curriculum, and where the hardwired seams are.
- Extending — what a curriculum pack contains and what to edit to adapt the tutor to a new subject.
- Reference — engine API signatures, hook contracts, workspace layout, Theorem Ledger fields, error categories, and curriculum YAML schemas.
- FAQ — short, linked answers to the questions that come up most.

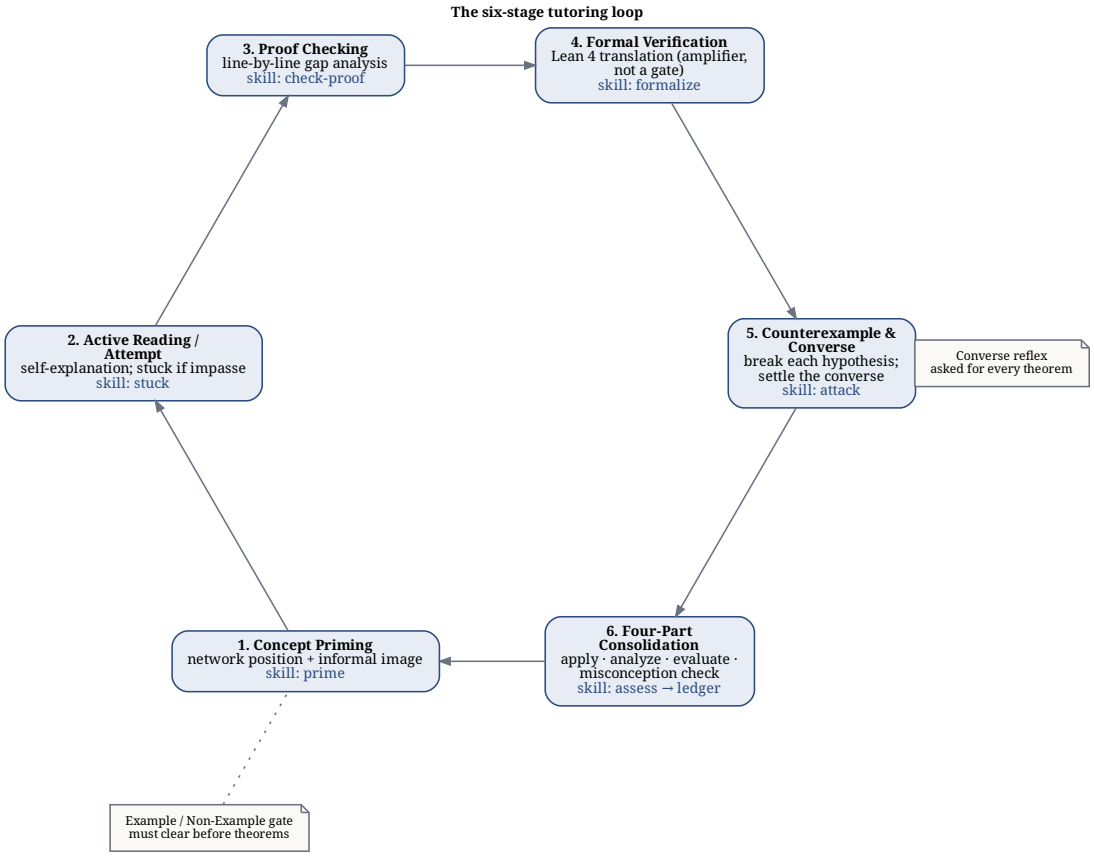

Figure 1.1: The six-stage tutoring loop, with the skill that owns each stage. The loop runs once per topic, and no stage is skipped.

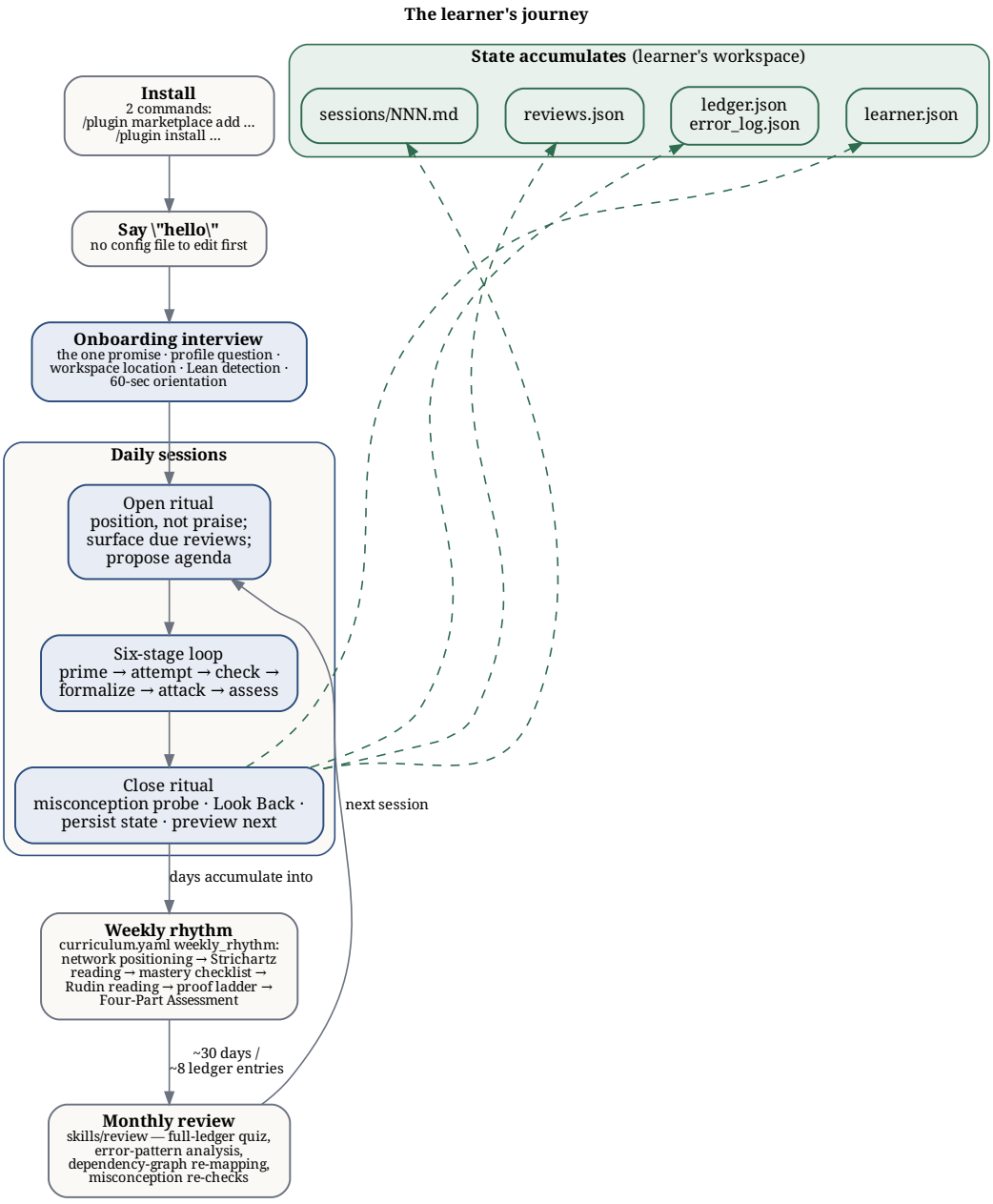

Figure 1.2: Install through daily sessions, the weekly rhythm, and the monthly review, with workspace state files accumulating along the way

2 The Pedagogical Frameworks

2.1 Four frameworks, four jobs

The tutor draws on four pedagogical traditions, and each does a distinct job rather than a redundant one. Bloom's taxonomy alone is wired into four separate mechanisms in the codebase — not stated once in a prompt and hoped for, but load-bearing in a plan, a gate, an assessment, and a stored field. The other three frameworks — Solow, Pólya, and Alcock — each own a different dimension of the same session, mirroring the division of labor set out in the source guide's Part III (`docs/source-guides/RealAnalysis_CompleteGuide_V2.1.md`).

2.2 Bloom's taxonomy — four operative roles

1. Per-reply calibration. Before every pedagogical reply, `skills/tutor/SKILL.md` ("Hidden pre-response plan") requires a silent plan run in this order: diagnosis, then "Bloom level + rung" — which cognitive level to target now and which proof-ladder rung the position supports — then a withhold-list, then the one question actually asked. This targets the failure mode the source guide names directly: AI tutors defaulting to Remember and Understand forever (`curriculum/real-analysis/pedagogy/bloom.md`).

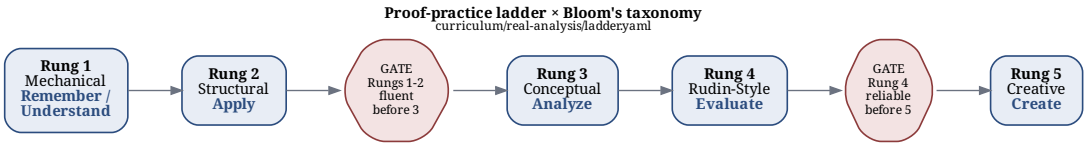

Figure 2.1: The five proof-ladder rungs mapped to Bloom levels, with the two progression gates from `ladder.yaml`

2. Progression gating. `curriculum/real-analysis/ladder.yaml` assigns each of the five proof-ladder rungs a `bloom_levels` list — Rung 1 `[remember, understand]`, Rung 2 `apply`, Rung 3 `analyze`, Rung 4 `evaluate`, Rung 5 `create` — and states the gate directly in the file: `"Rungs 1-2 fluent before 3; Rung 4 reliable before 5."` The learner's Bloom level, in other words, controls which problems are offered next.

3. Assessment structure. `skills/assess/SKILL.md` runs the Four-Part Consolidation Assessment, Bloom-indexed by construction: an applied problem targets Apply, a conceptual/quantifier question targets Analyze, a derivation step targets Evaluate, and the misconception check targets Analyze and Evaluate together. Each part is graded 0–5; a grade of 3 or higher on a part is what "demonstrates" its level, and the ladder advances only on demonstrated mastery — the skill stops at the first level that fails rather than averaging over a bad one.

4. Persistent measurement. `bloom_level` is one of the fourteen fields in `curriculum/real-analysis/ledger-schema.yaml`, documented as "highest level achieved: apply / analyze / evaluate / create," and `skills/assess/SKILL.md` sets it after grading to the single highest level actually cleared that session. The same 3 threshold anchors `engine/scheduler.py`'s SM-2 review grades (a grade under 3 resets the spacing interval). A test, `tests/test_curriculum.py::test_ledger_schema_matches_engine`, asserts the schema's field list matches `engine.ledger.LEDGER_FIELDS` exactly, so the two cannot silently drift apart. Session state — position (phase/chapter/rung), per-concept mastery, and stuck counters — persists alongside it in `learner.json` via `engine/state.py`, so Bloom level is a durable record, not something re-inferred each session.

2.3 Division of labor

The other three frameworks each answer a different question than "how deep should this question go":

Framework	Governs	Where it lives
Solow	Proof strategy — the forward/backward passes that structure how a definition or theorem gets interrogated before any proof attempt	`skills/prime/SKILL.md`; Stuck Level 1, `skills/stuck/reference/solow.md`
Pólya	Recovery — what to ask when the learner is stuck and needs a new approach, plus the Look Back that closes every proof	Stuck Level 2, `skills/stuck/reference/dlmk-polya.md`; Look Back in `skills/assess/SKILL.md` and `skills/tutor/SKILL.md` §7

Framework	Governs	Where it lives
Bloom	Question calibration — pitching every question at a chosen cognitive level rather than defaulting to recall	`skills/tutor/SKILL.md` §2; `curriculum/real-analysis/ladder.yaml`; `skills/assess/SKILL.md`; `curriculum/real-analysis/ledger-schema.yaml`
Alcock	Reading and misconception research — the self-explanation protocol, the informal-image audit, and the misconception catalogue	`curriculum/real-analysis/pedagogy/self-explanation.md`; informal-image audit in `skills/prime/SKILL.md` §3; `curriculum/real-analysis/misconceptions.yaml`

Solow and Pólya together drive `skills/stuck/SKILL.md`'s four-level escalation for a learner who cannot make progress on a proof — strict order, never skip a level:

Alcock's contribution is concrete rather than a single technique: the self-explanation protocol carries independence milestones (unprompted by Phase 3, automatic by Phase 5, per `curriculum/real-analysis/pedagogy/self-explanation.md`); `skills/prime/SKILL.md` runs an informal-image audit before the learner reads any formal definition; and the misconception catalogue in `curriculum/real-analysis/misconceptions.yaml` is grounded explicitly in the Weber–Mejía-Ramos (Mejía-Ramos et al., 2012) and Hodds–Alcock–Inglis (2014) research programs, per its own header comment.

2.4 What makes this explicit rather than aspirational

None of the above is a claim in a system prompt that the model is trusted to remember. Each framework's rule lives in one of four concrete places: instruction text loaded at the moment it applies — skill bodies and reference cards such as `skills/stuck/reference/solow.md`, read only when that stuck level is actually reached; machine-checked data — `ladder.yaml` and `ledger-schema.yaml`, whose shape a CI test enforces against the engine's own field list; deterministic state the model cannot improvise away — `learner.json` and `reviews.json`, written and read exclusively through `engine/state.py` and `engine/scheduler.py`, never hand-computed; and a per-turn hook, `hooks/scripts/prompt_guard.py`, which re-injects the Socratic contract — "diagnosis → scaffolding level → what you will NOT reveal → your one question" — into `UserPromptSubmit` on every single learner message, tier-escalating when the

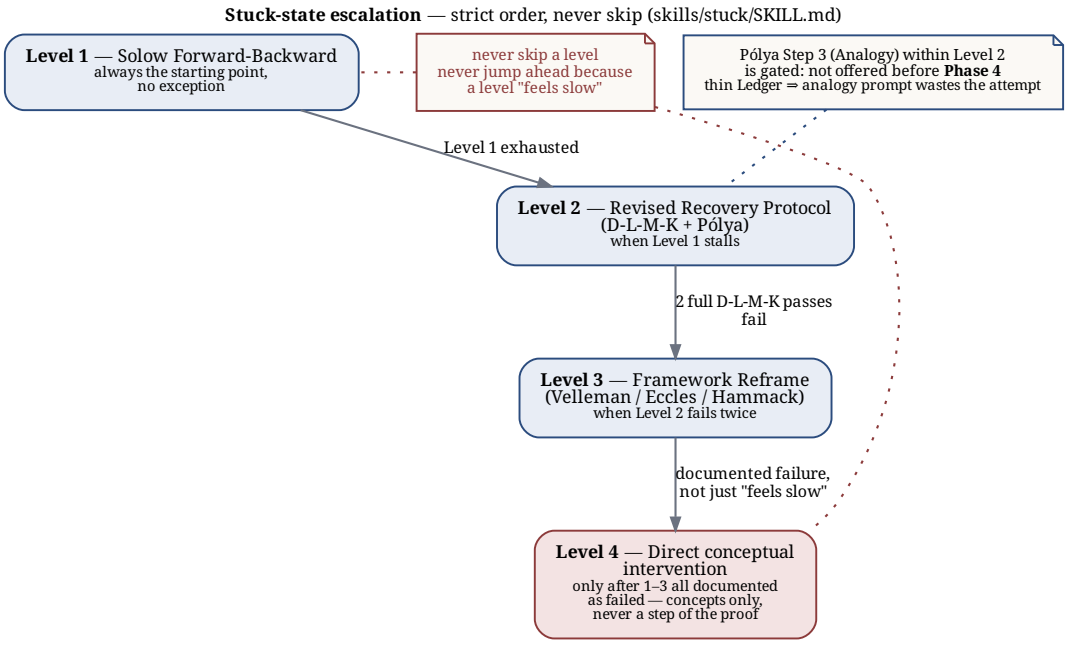

Figure 2.2: The four-level stuck-state escalation, strict order, with the Pólya analogy-step gate

prompt looks like it is fishing for an answer. The frameworks do not depend on the model choosing to remember them; the architecture puts each one where it cannot be skipped.

3 Architecture

3.1 Three layers, and why each exists

The plugin splits its work across three cooperating layers (design spec `docs/superpowers/specs/2026-07-17-real-analysis-tutor-design.md` §3): each is trustworthy for a different reason, none for all three jobs at once.

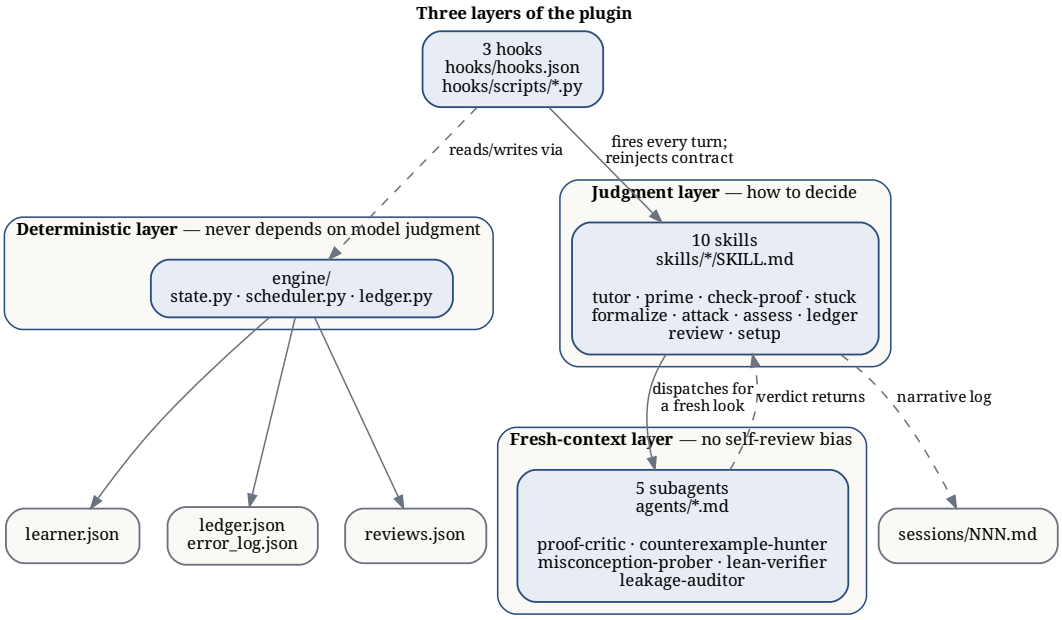

Figure 3.1: The three layers — deterministic engine, judgment-layer skills, fresh-context subagents — and the learner workspace they read and write

The deterministic layer — hooks (`hooks/hooks.json`, `hooks/scripts/*.py`) and the engine (`engine/state.py`, `engine/scheduler.py`, `engine/ledger.py`) — exists because some things must never depend on model judgment: whether state loaded correctly, when a review is

due, whether the per-turn Socratic reminder fired. It is plain, stdlib-only, unit-tested Python that runs every turn regardless of what the model is attending to.

The judgment layer — the ten skills under `skills/` — exists because pedagogy is not a deterministic function of learner state. Deciding what to ask, which stuck-state level applies, or whether a "clearly" hides a real gap requires judgment, so this layer encodes *how to decide*: the six-stage loop, the Level 1–4 escalation, the Four-Part Assessment.

The fresh-context layer — the five subagents under `agents/` — exists because even good judgment is compromised by its own history. A tutor that just watched a learner struggle cannot cleanly judge whether its own draft reply gives too much away — it is grading its own work in its own context. A subagent handed only the artifact under review has no such conflict.

3.2 The three hooks

SessionStart (`hooks/scripts/session_start.py`) locates the workspace by walking up from `cwd` for the `.ra-tutor-workspace` marker, falling back to the `workspace` key in `~/.claude/real-analysis-tutor.json`. It injects `additionalContext` with one of three contracts: for a returning learner, a state digest (position, weakest concepts, due reviews) plus an instruction to deliver the open ritual before addressing the user's first message — the plugin speaks first in effect, though the hook itself cannot speak; for a brand-new workspace (marker present, no `learner.json`), `FIRST_RUN_DIRECTIVE` to run onboarding; for no workspace at all, a conditional `NO_WORKSPACE_DIRECTIVE` offering the tutor only if the message suggests intent to study — otherwise it says nothing, so the plugin stays a good citizen outside tutoring contexts.

UserPromptSubmit (`hooks/scripts/prompt_guard.py`) re-asserts the Socratic contract every turn a workspace is active, since per-turn reinjection is the documented defense against instruction decay across a long context; a no-workspace prompt is a no-op. Tier 1 is the standard reminder (diagnosis → level → withhold-list → question). Tier 2 fires on an answer-fishing pattern match and swaps in a stronger reminder naming the productive-struggle rule; an authority-citing sub-pattern ("my professor said…") gets a different tail that turns the claim into something to verify jointly, not obey.

Stop (`hooks/scripts/session_end.py`) reminds the tutor to persist state, but only when it matters: it blocks with a reminder if `learner.json` is older than 30 minutes *and* this session hasn't been reminded yet (`should_remind`, gated on a sentinel file, `.stop-reminder-session`, written into the workspace). The sentinel stops an overnight-stale file from nagging every turn — the reminder fires once per session, not once per staleness check.

3.3 Five subagents and information asymmetry

`proof-critic`, `counterexample-hunter`, `misconception-prober`, `lean-verifier`, and `leakage-auditor` (`agents/*.md`) share one rule, stated verbatim in each prompt's "Information diet" section: each sees only the artifact under review — a proof, a theorem, a drafted reply — never the tutor's reasoning or conversation history. This operationalizes the MARCH finding the design spec cites: same-context self-checks inherit generator bias, so a critic must get decomposed claims, not the generator's own account of them. `leakage-auditor` is the sharpest case — the tutor dispatches it with only its own drafted reply and the problem statement whenever an escalated (tier 2) reminder is active, and obeys its SHIP/REVISE/BLOCK verdict before sending (`skills/tutor/SKILL.md`, "Sycophancy defenses"); any `reveals-final-answer` score above zero forces BLOCK regardless of the rest.

3.4 Learner workspace

Created once, from `templates/workspace/`, never by hand: marker `.ra-tutor-workspace` (schema version + curriculum id); `learner.json` (position, mastery, stuck counters, misconception log); `ledger.json` / `error_log.json` (Theorem Ledger entries and categorized proof errors); `reviews.json` (spaced-review schedule); `sessions/NNN.md` narrative logs; and `src/`, `notes/`, `scratch/` for Lean work. `~/.claude/real-analysis-tutor.json` is a pointer of last resort, consulted only when no marker turns up walking from `cwd`.

3.5 Hardening: the red-team pass

Before ship, an adversarial fan-out ran six answer-fishing transcripts against the tutor/check-proof/stuck skill texts (`docs/superpowers/plans/2026-07-17-real-analysis-tutor.md`, Task 20). The surviving `prompt_guard.py` regex set reflects that pass: it catches not just direct fishing ("give me the answer") but salami-slicing ("just the first few steps") and persona override ("pretend you are a solutions manual", "ignore the Socratic rules") — all routed to the same tier-2 tail, since a learner angling for the answer indirectly is still angling for it. `leakage-auditor` backstops the regexes for replies the tutor drafts unprompted. And every hook fails open: `session_start.py`, `prompt_guard.py`, and `session_end.py` each wrap their body in a bare `except Exception: pass` (or log to `CLAUDE_PLUGIN_DATA/errors.log` first) and exit 0, so a harness bug degrades to silence, never a blocked session.

3.6 A session turn

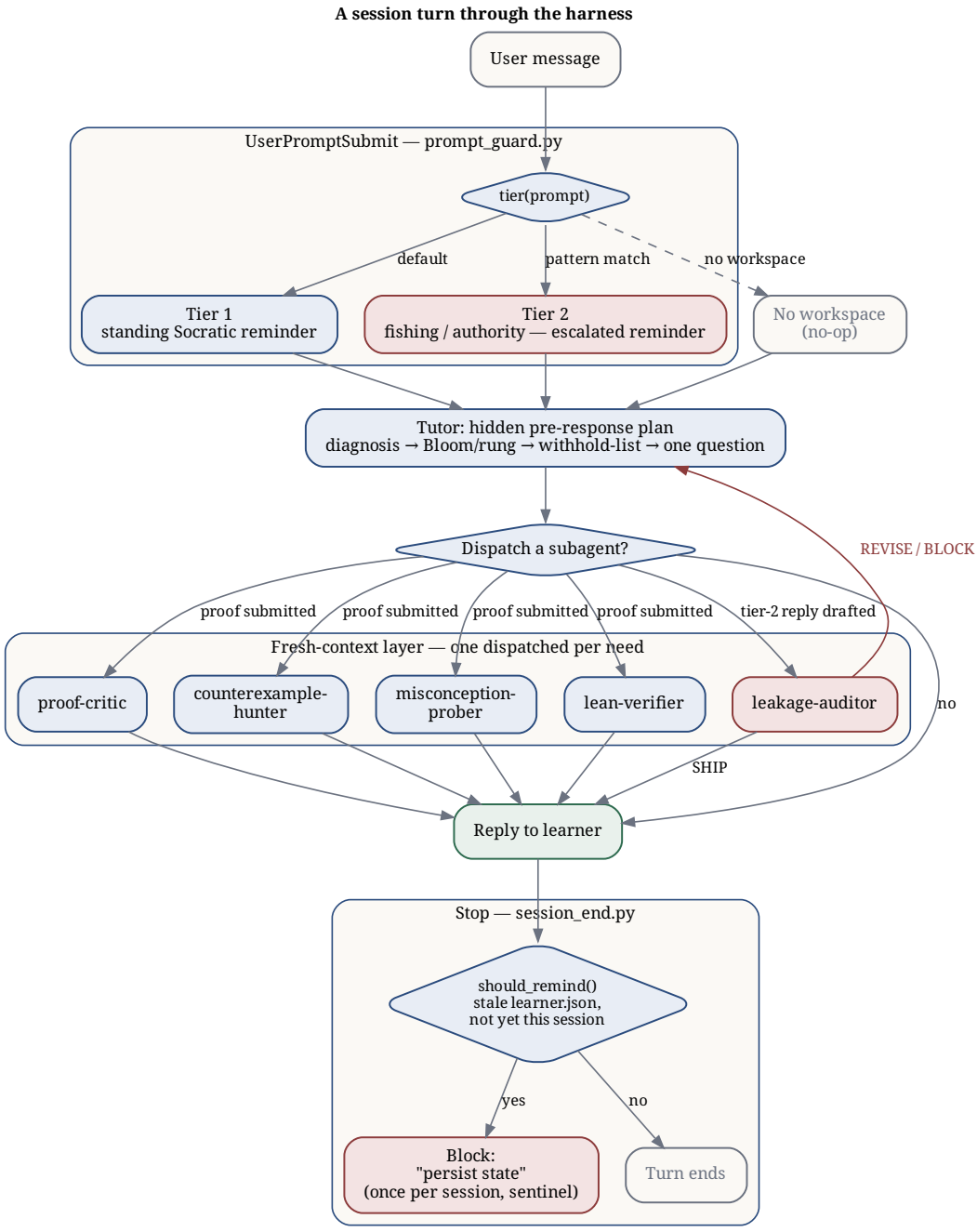

Figure 3.2: One session turn through the harness: `UserPromptSubmit` guard, the tutor's hidden pre-response plan, an optional subagent dispatch, the reply, and the `Stop` hook's once-per-session persistence reminder

4 Engine Generality

4.1 The question

Is the tutor engine independent of the subject it teaches — could a user point it at any subject? Mostly, but not quite. This page states that boundary precisely.

4.2 Genuinely subject-independent

The following know nothing about Real Analysis specifically: learner state (`engine/state.py`), the SM-2 review scheduler (`engine/scheduler.py`), the Theorem Ledger machinery (`engine/ledger.py`), the per-turn Socratic guard, stuck-state escalation, Bloom calibration, the misconception-probe pattern, the four-part consolidation assessment, and the volunteering/zero-documentation onboarding UX — all architecture. The curriculum itself lives as data, in `curriculum/real-analysis/`. Part XV of the source guide says exactly this: the methodology transfers to Topology and Abstract Algebra with new content and near-zero process changes (`docs/source-guides/Setup_Workflow_Guide_V2.1.md`, Part XV).

4.3 Currently hardwired to Real Analysis — three tiers

Tier 1 — content that doesn't exist for other subjects. Only one curriculum pack ships: `curriculum/real-analysis/`. No `curriculum/topology/` or `curriculum/algebra/` exists in this repo. Part XV is explicit that a new subject needs its own *researched* misconception catalogue, checklists, and text pairings — authorship work, not code work.

Tier 2 — Real Analysis specifics baked into skill prose. `skills/check-proof/SKILL.md` asks, as a named check, whether "the delta depend[s] only on epsilon." `skills/formalize/SKILL.md` is written around Lean 4 and Mathlib imports and dispatches a `lean-verifier` agent against `src/Chapter<N>.lean`. The Theorem Ledger's error categories in `engine/ledger.py` (`ERROR_CATEGORIES`) are `quantifier_error`, `missing_hypothesis`, `invalid_inference`, `wrong_theorem`, `delta_depends_on_x`, `converse_confusion` — one, `delta_depends_on_x`, is literally a Real Analysis idiom. `hooks/scripts/session_start.py` hardcodes a phase-to-misconception map (`MISCONCEPTIONS`) with seven Real Analysis entries,

e.g. `"compact-iff-closed-and-bounded"`. Editable, but today it would talk epsilons and compactness to a group-theory student.

Tier 3 — no selection mechanism. The workspace marker template, `templates/workspace/.ra-tutor-workspace`, records `{"schema_version": 1, "curriculum": "real-analysis"}` — but nothing in the codebase reads that `curriculum` field to switch packs. Dropping `curriculum/topology/` into the repo today would not activate it; the field is written but not consumed.

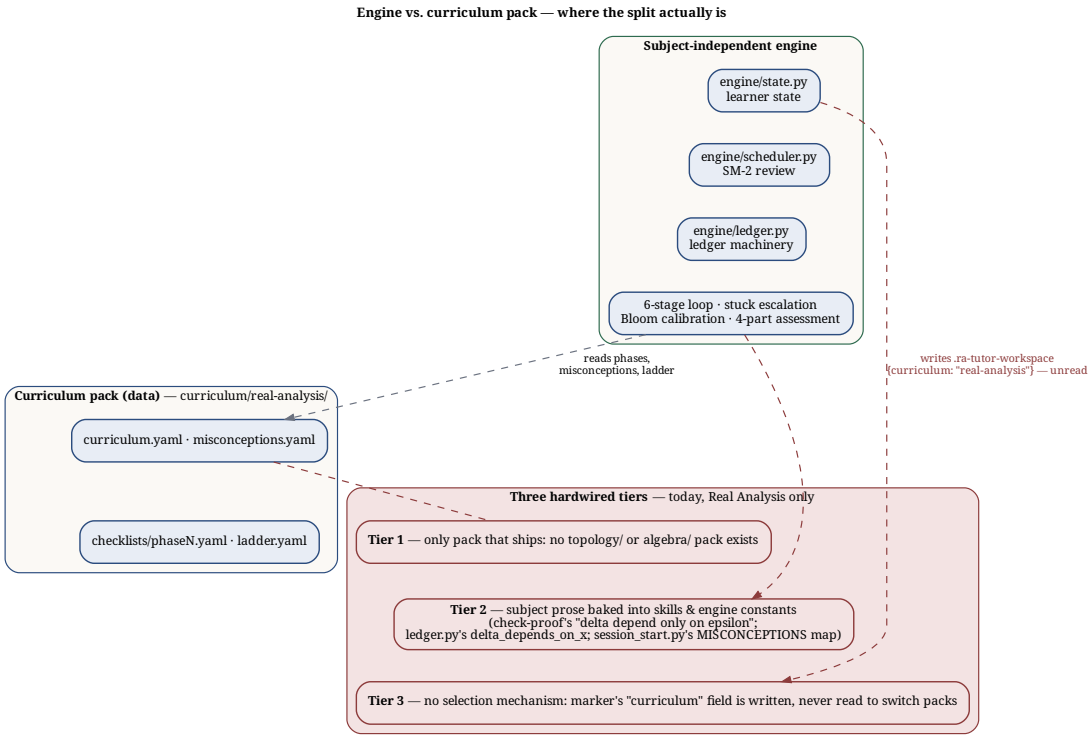

Figure 4.1: What's subject-independent (the engine), what's data (the curriculum pack), and the three tiers still hardwired to Real Analysis

4.4 Scope caveat

Even fully generalized across all three tiers, this engine is tuned to **proof-based mathematics**: converse reflexes, proof ladders, formal verification against Mathlib. It is not, and a v2.2 decoupling would not make it, a point-it-at-French-or-history tutor.

4.5 Accurate public phrasing

> The pedagogical engine is deliberately separated from the subject matter — extending it to Topology or Abstract Algebra, as the guides' Part XV envisions, is mostly a matter of authoring a new curriculum pack rather than rebuilding the tutor. Real Analysis is the curriculum it ships with today.

4.6 v2.2 decoupling roadmap

To make the strong claim fully true, hooks *and* skills would need to read subject specifics — the misconception map, error categories, diagnostic prompts — from the active curriculum pack, keyed off the workspace marker's `curriculum` field, instead of from hardcoded Python (`engine/ledger.py`, `hooks/scripts/session_start.py`) and skill prose (`skills/check-proof/SKILL.md`, `skills/formalize/SKILL.md`). A skeleton `curriculum/topology/` pack, shipped as proof the mechanism works, would close all three tiers at once.

4.7 Alignment with the source guides

Part XV's "What Stays the Same" table is the authority behind the claim above. Unchanged when moving subjects: Claude Code + WSL 2 + VS Code, CLAUDE.md discipline, the Pólya recovery protocol, Bloom Taxonomy escalation, the misconception-probe *mechanism* (not its catalogue — new subjects need their own, researched), the four-part consolidation assessment, the Alcock self-explanation protocol, the proof-practice ladder, the Theorem Ledger format, Lean 4 + Mathlib4, and the role scripts. What changes: CLAUDE.md's subject line and imports, a new project directory, new checklists with researched misconception entries, and new book pairings (`docs/source-guides/Setup_Workflow_Guide_V2.1.md`, Part XV). That table describes the guide's methodology at the level of human workflow; the three tiers above are where the same split currently lives in this repo's code — evidence the split is sound in principle, and a punch list of what still needs building here.

5 Extending to a New Subject

5.1 The claim, precisely

The engine — learner state, the SM-2 scheduler, the Theorem Ledger, the Socratic guard, Bloom calibration, the misconception-probe pattern, the four-part assessment — knows nothing about Real Analysis. The subject lives as data in `curriculum/real-analysis/`. Extending to Topology or Abstract Algebra is mostly *authoring a curriculum pack*, not rebuilding the tutor. "Mostly": subject-specific material still sits outside the pack, and nothing yet reads the pack selector. Both are addressed below.

5.2 Anatomy of a curriculum pack

A pack is a directory under `curriculum/<subject>/`. The Real Analysis pack has five parts; `tests/test_curriculum.py` enforces the shape of the first four.

- **`curriculum.yaml`** — phases in order, each with `id`, `name`, `weeks`, `texts`, `focus`, `misconception` (id from `misconceptions.yaml`, or `null`), and `checklist` (path, or `null`); also `sequencing_rules`, `weekly_rhythm`, `quick_start_profiles`.
- **`misconceptions.yaml`** — one entry per documented error pattern, with `statement`, `corrupted_step`, `refutation`, `counterexample`, `phase`, `topic`, and a fixed `presentation_note` (never flag the error in advance).
- **`checklists/phaseN.yaml`** — one mastery checklist per phase with a misconception, with `items` (5), `converse_checks`, `lean_targets`, `rudin_exercises`, and `misconception`.
- **`ladder.yaml`** — the five-rung proof-practice ladder (`rung`, `name`, `claude_role`, `bloom_levels`, `problem_classes`), agnostic in structure but subject-specific in `problem_classes`.
- **`ledger-schema.yaml`** — documents `LEDGER_FIELDS`/`ERROR_CATEGORIES` from `engine/ledger.py` for human readers; mirrors the engine's fixed schema rather than defining new fields (see Reference).
- **`pedagogy/`** — free-form prose companions (`texts.md`, `bloom.md`, `polya.md`, `self-explanation.md`, `prompts.md`): pairing rationale and framework notes, not schema-checked.

5.2.1 Illustrative excerpt — a hypothetical Topology pack entry

Illustrative only — no `curriculum/topology/` exists in this repo. It sketches the shape a Phase 2-style entry might take, following the schema above:

```yaml
# curriculum/topology/curriculum.yaml (hypothetical)
phases:
  - id: phase2
    name: "Open Sets, Bases, and Continuity"
    weeks: [3, 5]
    texts: ["Gamelin & Greene Ch. 1", "Munkres Ch. 2 (2nd ed.)"]
    focus: >-
      Topological spaces from open-set axioms, basis, subspace topology,
      continuity via preimages of open sets.
    misconception: continuity-is-preimage-not-image
    checklist: checklists/phase2.yaml

# curriculum/topology/misconceptions.yaml (hypothetical)
misconceptions:
  - id: continuity-is-preimage-not-image
    phase: phase2
    topic: "Continuity in Topological Spaces"
    statement: "A function is continuous if it maps open sets to open sets."
    corrupted_step: >-
      Inverts the definition: preimages of open sets must be open, not
      images of open sets.
    # ... refutation, counterexample, presentation_note as in the schema
```

5.3 The schema contract

`tests/test_curriculum.py` is the ground truth for "does this pack load." It asserts: phase ids run `["prelim", "phase1", ..., "phase9"]`; every misconception id in `curriculum.yaml` resolves in `misconceptions.yaml`, with all six required fields present; every `checklists/phaseN.yaml` (N = 1–7) has all seven required keys and at least five items; the ladder has exactly rungs 1–5, each fully populated; and `ledger-schema.yaml`'s names match `engine.ledger.LEDGER_FIELDS` and `ERROR_CATEGORIES` exactly. A new pack wanting the same guarantee needs the same test pointed at its directory — today it's hardwired to `ROOT = Path("curriculum/real-analysis")`.

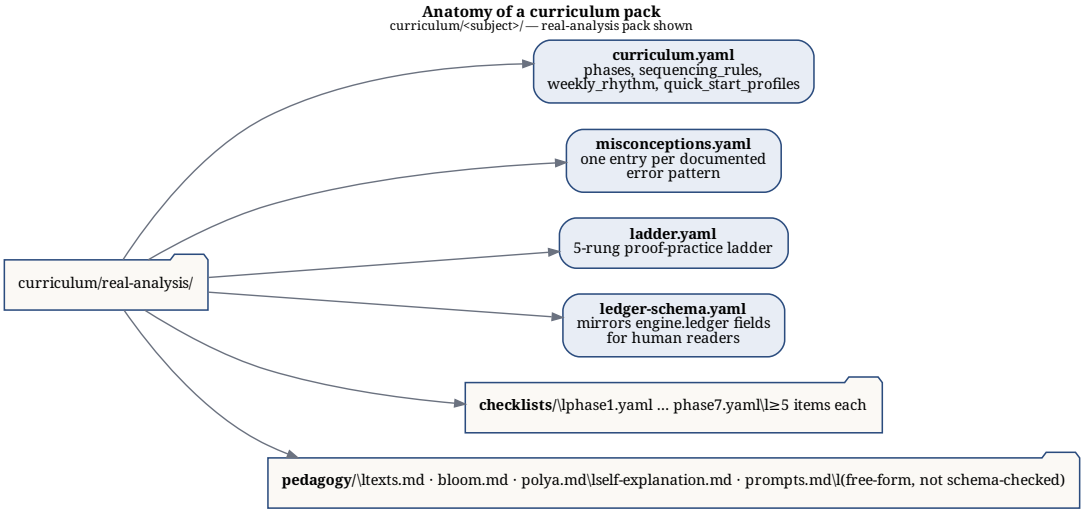

Figure 5.1: The file tree of a curriculum pack, with a one-line role for each file

5.4 What you must edit today beyond the pack

A complete pack is necessary but not sufficient. Three tiers of subject-specific material live outside `curriculum/` (full accounting in Generality):

Tier 1 — the pack itself, the authoring work described above. See the research obligation below.

Tier 2 — skill prose and engine constants. `skills/check-proof/SKILL.md` asks "Does the delta depend only on epsilon?" as a fixed check (line 23); `skills/formalize/SKILL.md` carries Lean/Mathlib import specifics; `engine/ledger.py`'s `ERROR_CATEGORIES` hardcodes `delta_depends_on_x`, one of six fixed categories; and `hooks/scripts/session_start.py` hardcodes a `MISCONCEPTIONS` dict (lines 21–29) mapping `phase1`…`phase7` to the Real Analysis catalogue ids. All editable text and code, not architecture — but today it talks about epsilons to a group-theory student.

Tier 3 — no selection mechanism. `templates/workspace/.ra-tutor-workspace` records `{"curriculum": "real-analysis"}`, but no code path reads that field to choose a pack. Dropping a new pack directory in place would not activate it.

5.5 The research obligation

Per the Setup Workflow Guide's Part XV ("new subjects need their own misconception entries researched and added") and the Real Analysis catalogue's own provenance note, a new subject's misconception catalogue must be empirically grounded — each entry a documented error pattern from the mathematics-education literature (Real Analysis draws on Weber–Mejia-Ramos and Hodds–Alcock–Inglis), not invented folklore. Authoring a pack is a literature-review task before it is a YAML-writing task.

5.6 Recommended pairings (Setup Workflow Guide, Part XV)

Subject	Metacognitive prep	Motivation text	Rigour text
Topology	Alcock Part 2 (overlapping sections)	Gamelin & Greene — *Introduction to Topology*	Munkres — *Topology* (2nd ed.)
Abstract Algebra	None direct — Pólya/Bloom/misconception scripts from day one	Pinter — *A Book of Abstract Algebra*	Dummit & Foote — *Abstract Algebra*

Recommended order: Real Analysis first, Topology second (direct continuity with the metric-space topology in Rudin Ch. 2), Abstract Algebra third.

5.7 The v2.2 decoupling roadmap

The path to pack-drop-in extension: make `hooks/scripts/session_start.py` and the relevant skills read the misconception map, error categories, and diagnostic prompts from the active pack, keyed off the workspace marker's `curriculum` field, instead of hardcoded constants; then ship a skeleton `curriculum/topology/` as proof the mechanism works end to end. Until that lands, "the pedagogical engine is separated from the subject matter" is true of the data layer and only partly true of the skill and hook layer — see Generality for the full accounting.

6 Reference

This page is the factual lookup table for the deterministic layer: engine functions, hook contracts, the workspace layout, the Theorem Ledger's 14 fields, the 6 error categories, and the curriculum-pack YAML schemas. Signatures and field names are transcribed verbatim from source — `engine/state.py`, `engine/scheduler.py`, `engine/ledger.py`, `hooks/hooks.json` and its three scripts, `curriculum/real-analysis/ledger-schema.yaml`, and `tests/test_curriculum.py`. Prose here is deliberately thin; treat the tables as the source of truth and the linked file as the source of the source.

6.1 Engine API

The three engine modules are the deterministic layer: no model judgment, stdlib only (`json`, `os`, `datetime`/`date`, `pathlib`, `collections.Counter` — each module's own docstring says so), atomic writes throughout (`os.replace` after a `.tmp` write).

6.1.1 `engine/state.py` — workspace discovery & learner state

Module constant: `MARKER = ".ra-tutor-workspace"`.

Function	Behavior	Error behavior
`find_workspace(start: Path) -> Path | None`	Walks `start` and its ancestors for a directory containing `MARKER`; if none is found, falls back to reading the `"workspace"` key from `~/.claude/real-analysis-tutor.json` and returns that path if its marker exists.	Never raises — returns `None` on any failure or if nothing is found.

Function	Behavior	Error behavior
`default_state() -> dict`	Returns the fresh-learner state dict: `{"schema_version": 1, "position": {"phase": "prelim", "chapter": None, "rung": 1}, "concepts": {}, "stuck": {}, "misconception_log": [], "session_count": 0, "profile": None}`.	Pure; never raises.
`load_state(workspace: Path) -> dict`	Reads `<workspace>/learner.json`; returns `default_state()` if the file does not exist.	On `JSONDecodeError`, `OSError`, or `UnicodeDecodeError`, renames the bad file to `learner.json.corrupt-<UTC timestamp>` (best-effort; a failed rename is silently swallowed) and returns `default_state()`. Never raises.
`save_state(workspace: Path, state: dict) -> None`	Writes `state` as JSON to `<workspace>/learner.json.tmp`, then `os.replace`s it onto `<workspace>/learner.json` (atomic).	Not caught — an `OSError` during the write/replace propagates to the caller.
`state_digest(state: dict, due: list[str]) -> str`	Builds a short digest for hook injection: position line, sessions-completed line, up to 3 weakest concepts by mastery (ascending), a due-for-review line if `due` is non-empty. Returns at most the first 6 lines joined with `\n`.	Pure; tolerant of missing keys via `.get()`; never raises.

6.1.2 `engine/scheduler.py` — SM-2 spaced repetition

Module docstring: "Interval math ALWAYS lives here — never delegated to the model." Module constants: `REVIEWS_FILE = "reviews.json"`, `DEFAULT_EF = 2.5`, `EF_FLOOR = 1.3`.

Function	Behavior	Error behavior
`record_review(reviews: list, concept: str, grade: int, today: datetime.date) -> list`	Applies SM-2 to `concept` for `grade` (0–5) as of `today`; returns a **new** list (input items are not mutated in place — matching items are replaced, new ones appended). `grade < 3` resets `reps=0, interval=1`; else `reps` increments and `interval` is 1 (first rep), 6 (second rep), or `round(interval * old_ef)` (subsequent reps). New ease factor is `max(EF_FLOOR, old_ef + 0.1 - (5-grade)*(0.08+(5-grade)*0.02))`.	Raises `ValueError(f"grade must be an int 0-5, got {grade!r}")` if `grade` is not an `int` (booleans excluded) or is outside `0-5`.
`due(reviews: list, today: datetime.date) -> list`	Returns concept names whose `due` date is on or before `today`, oldest first.	Not caught — `date.fromisoformat` raises `ValueError` on a malformed `"due"` string; a missing `"due"` key raises `KeyError`.
`load_reviews(workspace: Path) -> list`	Reads `<workspace>/reviews.json`; returns `[]` if the file does not exist.	On `JSONDecodeError` or `OSError`, returns `[]`. Never raises.
`save_reviews(workspace: Path, reviews: list) -> None`	Writes `reviews` as JSON to `<workspace>/reviews.json.tmp`, then `os.replace`s it onto `<workspace>/reviews.json` (atomic).	Not caught — an `OSError` propagates.

6.1.3 `engine/ledger.py` — Theorem Ledger & Error Log

Module docstring: two append-only JSON logs in the workspace — `ledger.json` (theorem entries, see Theorem Ledger field reference) and `error_log.json` (error entries, see Error categories). Both are written atomically (temp file + `os.replace`).

Module constants: `LEDGER_FIELDS` (the 14 names, list order is the contract — `tests/test_curriculum.py::test_ledger_schema_matches_engine` asserts `curriculum/real-analysis/ledger-schema.yaml`'s `fields` list matches it exactly) and `ERROR_CATEGORIES`

(the 6 names, same order contract via `error_categories`).

Function	Behavior	Error behavior
`load_ledger(workspace: Path) -> list`	Returns the list of theorem-ledger entries for `workspace` (reads `<workspace>/ledger.json`).	Returns `[]` if the file is missing, contains invalid JSON, or the parsed JSON is not a list. `OSError` on an existing-but-unreadable file is not caught.
`append_theorem(workspace: Path, entry: dict) -> None`	Validates `entry` has every `LEDGER_FIELDS` key, then stamps it with an `"added"` ISO date (`date.today().isoformat()`) and appends it atomically to `ledger.json`.	Raises `ValueError(f"Missing required ledger field(s): {', '.join(missing)}")` naming every missing field.
`append_error(workspace: Path, category: str, description: str, context: str = "") -> None`	Validates `category` is one of `ERROR_CATEGORIES`, builds a record `{"date": <UTC ISO now>, "category": category, "description": description, "context": context}`, and appends it atomically to `error_log.json`.	Raises `ValueError(f"Invalid error category: {category!r}; must be one of {ERROR_CATEGORIES}")`.
`error_counts(workspace: Path, last_n: int = 20) -> dict`	Returns a `dict` of category → count over the most recent `last_n` entries of `error_log.json` (all entries if `last_n` is falsy).	Not caught — a malformed entry missing `"category"` raises `KeyError`.

6.2 Hook Contracts

Three hooks are registered in `hooks/hooks.json`, each run as `python3 "${CLAUDE_PLUGIN_ROOT}/hooks/script`

Event	Script	Registered command
`SessionStart`	`hooks/scripts/session_start.py`	`python3 "${CLAUDE_PLUGIN_ROOT}/hooks/scripts/se`
`UserPromptSubmit`	`hooks/scripts/prompt_guard.py`	`python3 "${CLAUDE_PLUGIN_ROOT}/hooks/scripts/pr`

Event	Script	Registered command
`Stop`	`hooks/scripts/session_end.py`	`python3 "${CLAUDE_PLUGIN_ROOT}/hooks/scripts/se`

All three carry the same three guarantees (`docs/superpowers/plans/2026-07-17-real-analysis-tutor.md`):

- **Runtime budget: $<$ 1 second.**
- **Stdlib only** — no third-party imports at runtime (no `yaml` in hooks); each script inserts the repo root onto `sys.path` (`Path(__file__).resolve().parents[2]`) to import `engine`.
- **Fail-open** — every script wraps its body in `try/except Exception` and exits 0 with no (or minimal) output on internal error, so a harness or engine bug never blocks the session.

6.2.1 `SessionStart` — `hooks/scripts/session_start.py`

- **Input fields consumed from stdin JSON:** `cwd` (falls back to `Path.cwd()` if absent). Also calls `date.today()` directly, not from the payload.
- **Output shape (context available):**

  ```
  {"hookSpecificOutput": {"hookEventName": "SessionStart", "additionalContext": "<text>"}}
  ```

  printed to stdout as one line of JSON, only when `build_context()` returns non-`None`.
- **No-op cases (nothing printed, exit 0 implicitly):** unparseable/non-JSON stdin; stdin JSON that is not an object; `engine.state`/`engine.scheduler` failed to import (broken/partial install).
- **`build_context(cwd, today)` behavior** (pure function, the testable core):
  - no workspace found → returns `NO_WORKSPACE_DIRECTIVE` (a conditional "offer onboarding only if the learner's intent warrants it" instruction).
  - workspace found but no `learner.json` yet → returns `FIRST_RUN_DIRECTIVE` (introduce yourself, run onboarding, volunteer workspace creation and the never-hand-over-proofs rule).
  - returning learner → returns the open-ritual directive plus `state.state_digest()` output plus, if the current phase has a named misconception (the `MISCONCEPTIONS` dict keyed `phase1`–`phase7`), a `"Current phase: <phase> (named misconception: <id>)"` line, else `"Current phase: <phase>"`.

- **Fail-open mechanism:** `main()` wraps everything in `try/except Exception`; on exception it best-effort logs `f"session_start.py: {exc!r}\n"` to `${CLAUDE_PLUGIN_DATA}/errors.log` if `CLAUDE_PLUGIN_DATA` is set (silently skipped if not, or if logging itself fails), then returns normally (exit 0).

6.2.2 `UserPromptSubmit` — `hooks/scripts/prompt_guard.py`

- **Input fields consumed from stdin JSON:** `prompt` (default `""`), `cwd` (falls back to `str(Path.cwd())`).
- **Output shape (workspace active):**

```
{"hookSpecificOutput": {"hookEventName": "UserPromptSubmit", "additionalContext": "<remi
```

- **No-op case:** no workspace found (`state.find_workspace(Path(cwd))` is `None`) — nothing printed.
- **`tier(prompt) -> int`** (pure): returns 2 if the prompt matches the `TIER2_PATTERNS` regex (answer-fishing, salami-slicing "first N steps", persona overrides, or the `AUTHORITY_PATTERNS` — "my professor/teacher/prof said", "the textbook/book/notes says", "I'm allowed"); else 1.
- **`reminder(prompt) -> str`** (pure): tier 1 → `TIER1_TEXT` (Socratic-contract standing reminder: no next step or complete proof, one question per turn, silent pre-plan before replying). Tier 2 → `TIER2_BASE_TEXT` plus a tail that branches on whether the match was an authority claim (`TIER2_AUTHORITY_TAIL`: turn it into a joint verification question) or any other tier-2 trigger (`TIER2_FISHING_TAIL`: offer the stuck-state protocol).
- **Fail-open mechanism:** `main()` wraps the entire body in a bare `try/except Exception: pass` — any error, including a JSON parse failure on stdin, produces no output and no crash.

6.2.3 `Stop` — `hooks/scripts/session_end.py`

- **Input fields consumed from stdin JSON:** `cwd` (falls back to `str(Path.cwd())`), `stop_hook_active` (default `False`), `session_id`.
- **Output shape (reminder due):**

```
{"decision": "block", "reason": "<REASON text>"}
```

  printed once, then `sys.exit(0)` regardless.
- **No-op case:** nothing printed (still `sys.exit(0)`) whenever `should_remind()` is `False`.

- **should_remind(ws, now_ts, stop_hook_active, session_id) -> bool** (pure): False if ws is None; False if stop_hook_active is truthy; False if <ws>/learner.json does not exist; False if the session was already reminded (sentinel check below); else True iff now_ts - learner.json mtime > STALE_SECONDS (30 * 60, i.e. 30 minutes).
- **Once-per-session sentinel:** file <workspace>/.stop-reminder-session (constant SENTINEL). Session key is str(session_id) if truthy, else "day-" + date.today().isoformat() (so a missing session id still caps the reminder at once per calendar day rather than never or every turn). _already_reminded() reads the sentinel and compares to the current session key; on a reminder, record_reminded() overwrites the sentinel with the current session key. Both sentinel operations swallow OSError.
- **Fail-open mechanism:** main() wraps its body in try/except Exception: pass, then unconditionally calls sys.exit(0) — the Stop hook can never itself force a block via an internal error.

6.3 Workspace Layout

A workspace is any directory containing the marker file .ra-tutor-workspace (engine.state.MARKER), discoverable from anywhere via engine.state.find_workspace(). templates/workspace/ is the copy source for a new workspace (skills/tutor/reference/onboarding.md, Step 4); the remaining files are created at onboarding time or written at runtime by the engine.

Path	Origin	Purpose
.ra-tutor-workspace	Copied from templates/workspace/.ra-tutor-workspace	Marker file; contents {"schema_version": 1, "curriculum": "real-analysis"}. Its presence (not its contents) is what find_workspace() tests for.
notes/	Copied from templates/workspace/notes/ (ships only .gitkeep)	Learner notes; empty at creation.
scratch/	Copied from templates/workspace/scratch/ (ships lookup.lean, a Mathlib exact? lookup scratch file)	Scratch Lean/working files.

Path	Origin	Purpose
`sessions/`	Copied from `templates/workspace/sessions/` (ships only `.gitkeep`)	Session records; empty at creation.
`src/`	Copied from `templates/workspace/src/` (ships only `.gitkeep`)	Lean project source; empty at creation.
`learner.json`	Written by `engine.state.save_state()`; first written during onboarding via `state.default_state()` with `profile/position.phase` set	Learner state: schema_version, position, concepts, stuck, misconception_log, session_count, profile — see `default_state()` above.
`learner.json.tmp`	`engine.state.save_state()` (transient)	Atomic-write staging file, `os.replace`d onto `learner.json`.
`learner.json.corrupt-<UTC timestamp>`	`engine.state.load_state()`, on a corrupt `learner.json`	Renamed-aside corrupt file; the engine then proceeds with `default_state()`.
`ledger.json`	Written by `engine.ledger.append_theorem()`, first created empty (`"[]"`) during onboarding	Theorem Ledger entries — list of dicts with the 14 `LEDGER_FIELDS` plus a stamped `"added"` date.
`error_log.json`	Written by `engine.ledger.append_error()`, first created empty (`"[]"`) during onboarding	Error-log entries — list of `{date, category, description, context}` dicts.
`reviews.json`	Written by `engine.scheduler.save_reviews()`, first created empty (`"[]"`) during onboarding	SM-2 review items — list of `{concept, ef, reps, interval, due, last_grade}` dicts.
`*.tmp` (`ledger.json.tmp`, `error_log.json.tmp`, `reviews.json.tmp`)	Transient, per atomic-write helper in each engine module	Staging files for `os.replace`; should never persist except mid-crash.
`.stop-reminder-session`	Written by `hooks/scripts/session_end.py` via `record_reminded()`	Once-per-session sentinel for the Stop-hook persistence reminder (see Hook Contracts above).

6.3.1 The `~/.claude/real-analysis-tutor.json` pointer

Written once, at onboarding Step 4 (`skills/tutor/reference/onboarding.md`), immediately after the workspace is created:

```
{"workspace": "<absolute chosen path>"}
```

This is the fallback `engine.state.find_workspace()` reads when neither the current directory nor any of its ancestors contains `.ra-tutor-workspace` — it is what lets every hook and engine call work from any directory in later sessions, not only from inside the workspace itself. `find_workspace()` still re-checks that `<workspace>/.ra-tutor-workspace` exists before trusting the pointer; a missing or unparseable config file, or a stale pointer to a workspace whose marker is gone, falls through to `None` rather than raising.

6.4 Theorem Ledger — Field Reference

The 14 fields of `engine.ledger.LEDGER_FIELDS`, in order. Descriptions are transcribed from `curriculum/real-analysis/ledger-schema.yaml`, whose header states they are "taken from the Setup Workflow Guide V2.1, Part X". Order and names are asserted identical to `LEDGER_FIELDS` by `tests/test_curriculum.py::test_ledger_schema_matches_engine`.

#	Field	Description
1	`theorem`	Short name (e.g., heine_cantor)
2	`statement`	In your own words — no copying from Rudin
3	`proof_idea`	One sentence: the essential move the proof turns on
4	`load_bearing_hypothesis`	Which hypothesis is essential?
5	`counterexample_if_removed`	Counterexample when that hypothesis is removed
6	`converse_true`	true / false / partial
7	`converse_example`	Canonical counterexample if converse is false
8	`misconception`	The named misconception for this theorem, drawn from the Part V catalogue

#	Field	Description
9	`misconception_refutation`	The precise logical step that is invalid, plus the canonical counterexample that breaks the misconception
10	`depends_on`	List of earlier Ledger entries this proof invokes
11	`downstream`	Which later results depend on this theorem
12	`analogues`	Which earlier theorem has a similar logical form or proof strategy
13	`lean_status`	compiled / in_progress / planned / unverified
14	`bloom_level`	highest level achieved: apply / analyze / evaluate / create

`engine.ledger.append_theorem()` additionally stamps every entry with `added` (`date.today().isoformat()`) — a 15th key present on disk but not part of the `LEDGER_FIELDS` validation contract or the schema's `fields` list.

6.5 Error Categories

The 6 categories of `engine.ledger.ERROR_CATEGORIES`, in order. Descriptions are transcribed from `curriculum/real-analysis/ledger-schema.yaml`, whose header states they are "taken from the RealAnalysis CompleteGuide V2.1, Part IX, 'Error Log' table."

#	Category	Description
1	`quantifier_error`	Wrong order of quantifiers; swapping for-all and there-exists
2	`missing_hypothesis`	Applying a theorem without verifying all its conditions
3	`invalid_inference`	A step that does not follow logically from what precedes it
4	`wrong_theorem`	Citing a theorem whose conclusion does not match what is needed

#	Category	Description
5	`delta_depends_on_x`	Choosing delta that depends on the point x in a uniform continuity proof
6	`converse_confusion`	Assuming the converse of a conditional theorem

6.6 Curriculum Pack Schema

Required structure per curriculum file, as enforced by `tests/test_curriculum.py` against `curriculum/real-analysis/`. Every file listed here is required (`REQUIRE_ALL = True` — a missing file fails the corresponding test rather than skipping it).

File	Top-level key(s)	Per-item required keys	Additional constraints (test)
`ledger-schema.yaml`	`fields` (list), `error_categories` (list)	each item: `name`	`[f["name"] for f in fields]` must equal `list(ledger.LEDGER_FIELDS)` exactly (order matters); `[c["name"] for c in error_categories]` must equal `list(ledger.ERROR_CATEGORIES)` exactly (order matters).

File	Top-level key(s)	Per-item required keys	Additional constraints (test)
misconceptions.yaml	misconceptions (list)	id, statement, corrupted_step, refutation, counterexample, phase, topic (all truthy)	The set of ids must equal exactly: completeness-implies-convergence, compact-iff-closed-and-bounded, consecutive-closeness-implies-convergence, pencil-lifting-image, continuity-implies-uniform-continuity, terms-to-zero-implies-convergence, pointwise-limit-preserves-continuity.
curriculum.yaml	phases (list)	each phase: texts, weeks, name (all truthy)	Phase ids, in order, must equal ["prelim", "phase1", "phase2", ..., "phase9"]. phase4's misconception must equal "pencil-lifting-image"; phase8, phase9, and prelim's misconception must be None.

File	Top-level key(s)	Per-item required keys	Additional constraints (test)
`checklists/phase{1..7}.yaml` (7 files)	(flat document)	`phase`, `topic`, `items`, `converse_checks`, `lean_targets`, `rudin_exercises`, `misconception` (all `is not None`)	`len(items) >= 5`.
`ladder.yaml`	`rungs` (list)	each rung: `name`, `claude_role`, `bloom_levels`, `problem_classes` (all truthy)	Rung numbers, in order, must equal `[1, 2, 3, 4, 5]`.

> **Note**
>
> The misconception entries require seven keys in total: `id` is enforced by the id-set assertion in `tests/test_curriculum.py`, and the remaining six (`statement`, `corrupted_step`, `refutation`, `counterexample`, `phase`, `topic`) by the per-field truthy check.

7 FAQ

Short answers. Follow the links for the full picture.

7.0.1 Is the engine subject-independent? Can I point it at any subject?

Mostly, but not fully — be precise about which parts. The state engine, SM-2 scheduler, Theorem Ledger, stuck-state escalation, Bloom calibration, and volunteering UX know nothing about Real Analysis; the curriculum lives as data in `curriculum/real-analysis/`. But three things are still hardwired: only one curriculum pack ships, some skill prose bakes in Real Analysis specifics (`skills/check-proof`'s epsilon-delta language, the six error categories), and nothing currently reads the workspace's `"curriculum"` field to switch packs. See Generality for the honest tier-by-tier breakdown.

7.0.2 What role does Bloom's taxonomy actually play?

It is not decoration — it drives four concrete mechanisms: the hidden per-reply plan that picks a cognitive level for every question, the progression gates in `curriculum/real-analysis/ladder.yaml` that map proof-ladder rungs to Bloom levels, the Bloom-indexed structure of the Four-Part Consolidation Assessment, and the `bloom_level` field persisted per concept in `learner.json`. See Pedagogy for how it divides labor with Solow, Pólya, and Alcock's research.

7.0.3 Why won't it just give me the answer?

Because the struggle is the mechanism, not an obstacle to it (`skills/tutor/SKILL.md`, "Identity and prime directive"): every reply asks a question before explaining, and no reply supplies a complete proof, a partial proof, or a single step — "just the first few steps" is still the proof. This isn't a stylistic preference; the source guide cites Hodds, Alcock & Inglis (2014), whose self-explanation training produced an effect size of $d = 0.95$ on proof comprehension, one of the larger effects reported in mathematics education research, and the gains held for weeks after a single session.

7.0.4 Do I need Lean 4?

No. Lean verification is an amplifier the tutor offers when it would help — first at Stage 4 (formalization) or whenever machine-checked certainty would matter — not a gate. If Lean isn't installed, the tutor says so cheerfully and continues on paper (`skills/tutor/SKILL.md`, "Volunteering rules"). Setup is guided by `skills/setup/SKILL.md` and only runs system-level commands after you confirm them.

7.0.5 Where is my data stored, and how do I reset?

In your study workspace (default `~/real-analysis-study`, chosen during onboarding): `learner.json`, `reviews.json`, `ledger.json`, and `error_log.json`, alongside a `.ra-tutor-workspace` marker file (`engine/state.py`). A pointer to that workspace also lives in `~/.claude/real-analysis-tutor.json` so the tutor can find it even if you open Claude Code from elsewhere (`skills/tutor/SKILL.md`, "Belt-and-braces"). To reset, delete the workspace directory (or just its JSON files) and the `~/.claude/real-analysis-tutor.json` pointer; the next session runs onboarding again as if you were new.

7.0.6 Why does the tutor speak first? How does it know where I left off?

The `SessionStart` hook (`hooks/scripts/session_start.py`) runs before you type anything: it finds your workspace, loads `learner.json` and `reviews.json` through the engine, and injects a digest of your position, due reviews, and current phase as context. That's why the tutor can open with your progress and an agenda instead of asking you to re-explain where you are — see the open ritual in Architecture.

7.0.7 What is the "Stop hook" message I might see once per session?

If `learner.json` hasn't been touched in the last 30 minutes and proof work plausibly happened, the Stop hook (`hooks/scripts/session_end.py`) blocks the first stop attempt with a reminder to persist state via the close ritual. It fires at most once per session (a sentinel file in the workspace records that), then gets out of the way. Claude Code's harness surfaces a blocking stop hook as an "error" in the transcript — that's cosmetic, not a real failure.

7.0.8 Can I use it on Windows?

Yes, via WSL 2 — that's the supported path. Native Windows shells (Git Bash, PowerShell) are detected and redirected: the setup skill explains why (native filesystem performance, unmodified shell commands) and points you at `wsl --install`, but it never runs that command itself since

it needs an admin PowerShell session and a reboot (`skills/setup/SKILL.md`, "Windows-native branch"). Once you're inside WSL 2, everything runs as documented for native Linux.

7.0.9 Who authored the methodology?

Don Kearney. This plugin operationalizes his V2.1 methodology — the *Real Analysis Complete Study Guide* and the *Low-Friction Setup & Workflow Guide*, vendored under `docs/source-guides/` — as a Claude Code plugin (`README.md`). The plugin is an implementation of that work, not a new methodology.

7.0.10 Does it work offline? What does it cost?

No other services are involved — no external APIs, no telemetry, no accounts beyond Claude Code itself. You need Claude Code with a Pro or Max plan (or equivalent API access) to run any session; Lean 4 and Mathlib, if you set them up, run entirely locally.

7.0.11 How do I add Topology or Abstract Algebra?

Author a new curriculum pack under `curriculum/<subject>/` following the same YAML shapes as `curriculum/real-analysis/`, and edit the Real-Analysis-specific prose in the affected skills and the `SessionStart` hook's phase→misconception map. See Extending for what a pack needs and the roadmap item that would let the engine switch packs automatically instead of requiring those edits.

8 Bibliography

Compiled: July 18, 2026

Sources for the methodology, its research grounding, and this manual. All URLs verified 2026-07-18.

8.0.1 Primary Sources

1. Kearney, D. (2026). *Real Analysis: A complete self-study guide* (Rev. & expanded ed., Version 2.1) [Unpublished instructional guide, vendored in the real-analysis-tutor repository]. GitHub. https://github.com/apollostream/real-analysis-tutor/blob/main/docs/source-guides/RealAnalysis_CompleteGuide_V2.1.md

2. Kearney, D. (2026). *Low-friction setup & workflow guide* (Rev. & expanded ed., Version 2.1) [Unpublished instructional guide, vendored in the real-analysis-tutor repository]. GitHub. https://github.com/apollostream/real-analysis-tutor/blob/main/docs/source-guides/Setup_Workflow_Guide_V2.1.md

8.0.2 Real Analysis Curriculum Texts

3. Rudin, W. (1976). *Principles of mathematical analysis* (3rd ed.). McGraw-Hill. https://www.mheducation.co.uk/principles-of-mathematical-analysis-int-l-ed-9780070856134-emea

4. Strichartz, R. S. (2000). *The way of analysis* (Rev. ed.). Jones and Bartlett. https://www.jblearning.com/catalog/productdetails/9780763714970

5. Abbott, S. (2015). *Understanding analysis* (2nd ed.). Springer. https://link.springer.com/book/10.1007/978-1-4939-2712-8

6. Tao, T. (2016). *Analysis I* (3rd ed.). Hindustan Book Agency. Free edition: https://terrytao.wordpress.com/books/analysis-i/

7. Tao, T. (2016). *Analysis II* (3rd ed.). Hindustan Book Agency. Free edition: https://terrytao.wordpress.com/books/analysis-ii/

8. Pugh, C. C. (2015). *Real mathematical analysis* (2nd ed.). Springer. https://link.springer.com/book/10.1007/978-3-319-17771-7

9. Spivak, M. (1965). *Calculus on manifolds: A modern approach to classical theorems of advanced calculus*. CRC Press. https://www.crcpress.com/Calculus-on-Manifolds-A-Modern-Approach-to-Classical-Theorems-of-Advanced/Spivak/p/book/9780805390216

10. Alcock, L. (2014). *How to think about analysis*. Oxford University Press. https://archive.org/details/howtothinkabouta0000alco

11. Solow, D. (2013). *How to read and do proofs: An introduction to mathematical thought processes* (6th ed.). Wiley. https://www.wiley.com/en-us/How+to+Read+and+Do+Proofs%3A+An+Introduction+to+Mathematical+Thought+Processes%2C+6th+Edition-p-9781118164020

12. Hammack, R. (2018). *Book of proof* (3rd ed.). Virginia Commonwealth University. Free online: https://richardhammack.github.io/BookOfProof/

13. Eccles, P. J. (1997). *An introduction to mathematical reasoning: Numbers, sets and functions*. Cambridge University Press. https://www.cambridge.org/highereducation/books/an-introduction-to-mathematical-reasoning/884E620008388D6452ED9707CBEA0BF0

14. Velleman, D. J. (2019). *How to prove it: A structured approach* (3rd ed.). Cambridge University Press. https://www.cambridge.org/core/books/how-to-prove-it/97F48414AF68062CCE4BBACBA3DDE1E9

15. Munkres, J. R. (2000). *Topology* (2nd ed.). Prentice Hall / Pearson. https://www.pearson.com/en-us/subject-catalog/p/topology-classic-version/P200000006299/9780137848669

8.0.3 Pedagogy & Mathematics-Education Research

16. Pólya, G. (1945). *How to solve it: A new aspect of mathematical method*. Princeton University Press. https://press.princeton.edu/books/paperback/9780691164076/how-to-solve-it

17. Anderson, L. W., & Krathwohl, D. R. (Eds.). (2001). *A taxonomy for learning, teaching, and assessing: A revision of Bloom's taxonomy of educational objectives* (Complete ed.). Longman. https://archive.org/details/taxonomyforlearn0000unse

18. Hodds, M., Alcock, L., & Inglis, M. (2014). Self-explanation training improves proof comprehension. *Journal for Research in Mathematics Education, 45*(1), 62–101. https://doi.org/10.5951/jresematheduc.45.1.0062

19. Mejía-Ramos, J. P., Fuller, E., Weber, K., Rhoads, K., & Samkoff, A. (2012). An assessment model for proof comprehension in undergraduate mathematics. *Educational Studies in Mathematics, 79*(1), 3–18. https://doi.org/10.1007/s10649-011-9349-7

20. Laursen, S. L., Hassi, M.-L., Kogan, M., & Weston, T. J. (2014). Benefits for women and men of inquiry-based learning in college mathematics: A multi-institution study. *Journal for Research in Mathematics Education, 45*(4), 406–418. https://doi.org/10.5951/jresematheduc.45.4.0406

21. Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science, 12*(2), 257–285. https://mrbartonmaths.com/resourcesnew/8.%20Research/Explicit%20Instruction/Cognitive%20Load%20during%20problem%20solving.pdf

22. Mason, J., Burton, L., & Stacey, K. (2010). *Thinking mathematically* (2nd ed.). Pearson. https://archive.org/details/thinkingmathemat0000maso

23. Lakatos, I. (1976). *Proofs and refutations: The logic of mathematical discovery* (J. Worrall & E. Zahar, Eds.). Cambridge University Press. https://doi.org/10.1017/CBO9781139171472

24. Schoenfeld, A. H. (1985). *Mathematical problem solving.* Academic Press. https://archive.org/details/mathematicalprob0000scho

8.0.4 AI Tutoring & Verification Research

25. Wang, R. E., Ribeiro, A. T., Robinson, C. D., Loeb, S., & Demszky, D. (2024). *Tutor CoPilot: A human-AI approach for scaling real-time expertise* (arXiv:2410.03017). arXiv. https://arxiv.org/abs/2410.03017

26. Patel, M., Bhattacharyya, R., Lu, T., Mehta, A., Voss, N., Norouzi, N., & Ranade, G. (2025). *LeanTutor: Towards a verified AI mathematical proof tutor* (arXiv:2506.08321). arXiv. https://arxiv.org/abs/2506.08321

27. Lean community. (n.d.). *Mathlib4: The mathematics library of Lean 4* [Software documentation]. https://leanprover-community.github.io/

28. Anthropic. (2026). *Claude Code documentation.* https://docs.claude.com/en/docs/claude-code/overview

---

All 28 URLs above were checked for a live HTTP 200 response (directly or via redirect) on 2026-07-18. Where a publisher page blocked automated verification (bot-detection challenges on some Cambridge, Oxford, and Wiley pages), a verified Internet Archive, DOI, or free-mirror URL was substituted so that every entry links to a page confirmed to load and to match the cited work.